

cs336 Assignment 1

2: Byte Pair Encoding Tokenizer

2.1 Unicode Standard

What Unicode character does `chr(0)` return?

For reference, `chr(97)` returns `a`, meaning that the character `a` corresponds to the code point `97`. `chr(48)` returns `0`.

`chr(0)` returns `'\x00'`. This is the character that corresponds to which to code point `0`. In python, `\x` indicates that the character is being described in hexadecimal by the following two numbers (`00`). So the question is, what does `\x00` actually represent? It must not be a standard character.

How does the character's string representation differ from printed representation?

`chr(0).__repr__()` returns `"'\x00'"`. This is telling python how to represent the escape sequence, so that it shows the escape sequence that would trigger the special character, instead of actually triggering what the special character represents. When we do `print(chr(0).__repr__())`, we get `'\x00'`.

What happens when this character occurs in text?

Outside of a print statement, it retains its escape sequence nature, and is combined with a string: `"this is a test" + chr(0) + "string"` returns `'this is a test\x00string'`. It's treating the contents inside the quotes as just another string. However, when we print this, i.e. `print('this is a test\x00string')`, we get `this is a test string`. Since the print statements are sending the actual bytes to the terminal, they are getting interpreted correctly.

It would appear that `\x00` represents the null character.

2.2 Unicode Encodings

Why would we prefer training our tokenizer on UTF-8 instead of UTF-16 or UTF-32?

Using UTF-16 or UTF-32 means we'd need more information to store even our basic letters compared to UTF-8. In UTF-8, most of the most common characters (english letters) are represented by just one byte. We prefer this since we are dealing with such large quantities of English text, and thus would need to store fewer bytes.

Why is the provided function incorrect?

Any example that involves a character that is encoded by more than one byte will fail. The bytes `b'\xe7\x89\x9b'` which encode the japanese kanji for 'cow' will cause this function to fail. Some bytes cannot be separated; the first byte itself never occurs on its own, and must always be followed by another byte. For it to appear alone results in an error. See below for more info.

Give a two byte sequences that does not decode to any Unicode characters.

`'牛'.encode()` returns `b'\xe7\x89\x9b'`, which means `b'\xe7\x89\x9b'.decode()` returns `'牛'`. But if we try taking these first two bytes, `b'\xe7\x89'.decode()` we get the error: `UnicodeDecodeError: 'utf-8' codec can't decode bytes in position 0-1: unexpected end of data`. This is because the first two bytes indicate that a third byte will follow.

2.5 TS and OWT Tokenizers

skipped

2.7 Tokenizer Experiments

skipped

3. Transformer LM Architecture

3.6 Full Transformer LM Resource Accounting

skipped

4: Training a Transformer LM

4.1 SGD Optimizer Learning Rate

How does learning rate affect things?

skipped

4.3 Adam W Accounting

How much peak memory does running AdamW require?

What is the maximum batch size you can use and still fit within 80GB memory?

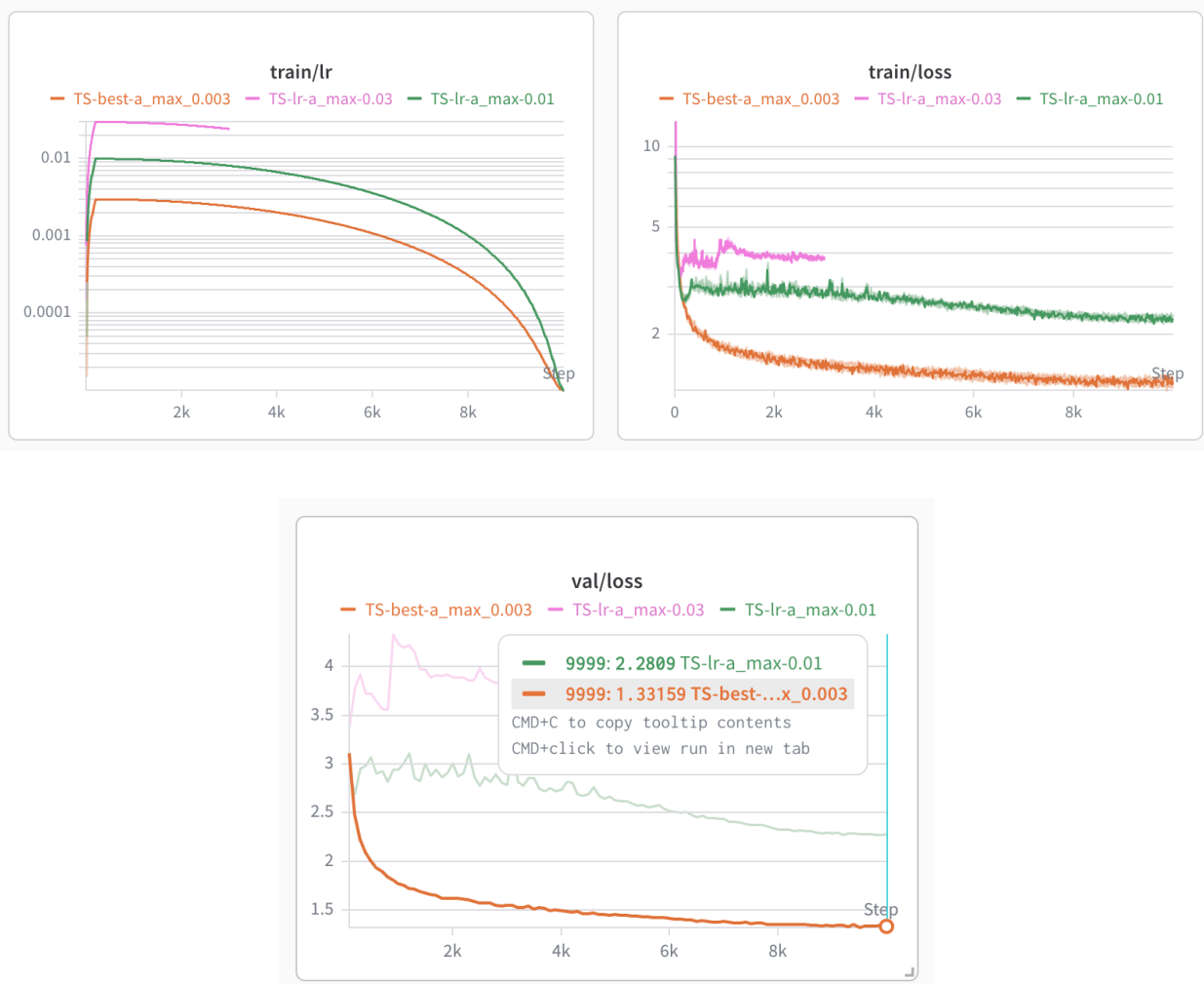
How many FLOPs does running one step of AdamW take?

Assuming you are able to get 50% MFU, how long would it take to train a GPT-2 XL for 400K steps and a batch size of 1024 on a single A100?

skipped

7: Experiments

7.2 Learning Rate



- a. Given the base parameters, we got a training loss of 1.33  with an `a_max` of 0.003. We found both 0.01 and 0.03 to be too unstable, and the runs diverged. Our search strategy was basically try a few different learning rates until 3k steps (30% of the dataset) to see if the runs remained stable. We wanted to be “aggressive” so that the model learned faster and converged on a lower loss, but without ending up diverging, as we saw in two of the cases here. We not that the validation loss looks equally good. We ended up running the best run all the way (orange) to end with the training loss.

Also, we set `a_min` as well as `T_w` and `T_c`. We set the warmup to 2% of the full dataset (in the llama paper they did 0.8%). We also note that in the llama paper, they set `a_min` to 10% of `a_max`. In this case, we may have gone too conservative, settling on a value of 0.00001, or 0.3% of the value of `a_max`, meaning that towards the end the model wasn’t really learning. We computed `T_c` based on the total number of iterations we would need to fit this formula: $\text{total tokens} = \text{batch_size} * \text{context_len} * \text{iterations}$.

(Other hyper params: `a_max`:0.003, `a_min`:0.00001, `batch_size`:128, `checkpoint_dir`:"./checkpoints", `checkpoint_prefix`:"run-2352bf0f7-checkpoint-", `context_length`:256, `d_ff`:1,344, `d_model`:512, `dataset`:"data/TinyStoriesV2-GPT4-train.npy", `device`:"cuda", `eps`:0.000001, `iterations`:10,000, `load_checkpoint_from`:null, `lr`:0.0001, `max_l2_norm`:1, `num_heads`:16, `num_layers`:4, `save_every`:2,000, `T_c`:10,000, `T_w`:200, `theta`:10,000, `val_dataset`:"data/TinyStoriesV2-GPT4-valid.npy", `val_every`:100, `vocab_size`:10,000, `weight_decay`:0.01)

- b. Above we are lucky to see two divergent cases and one non-diverging. The final value we set was an order of magnitude below the diverging values, inline with the folk wisdom. In both cases, as the `a_max` (i.e. learning rate) gets larger, the loss seems to diverge earlier and plateau at a higher level. It’s hard to say if this will necessarily happen every time, but I think it’s safe to say that when the `lr` gets larger, the chance that the model diverges gets higher and higher. As to what this means, or why this happens, I’m not really sure. If I had to guess, I’d say it has something to do with the fact that the model is taking steps that are too large, and ends up in a local minimum that it can no longer escape.

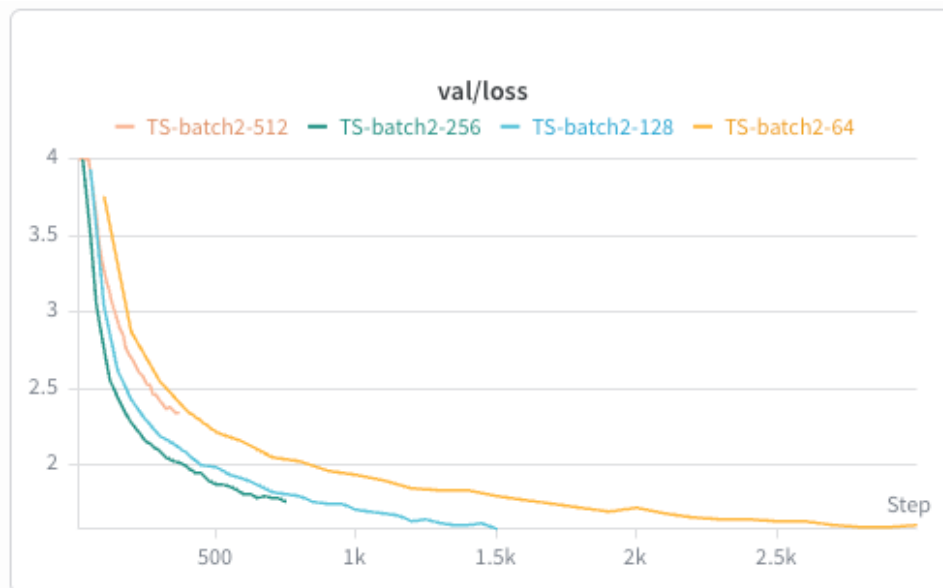
Claude Feedback: The loss doesn't settle in a "local minimum it can't escape" — it's the opposite. The steps are so large that the optimizer **overshoots** the good minima and bounces around high-loss regions. It's like trying to land on a narrow valley by taking giant leaps.

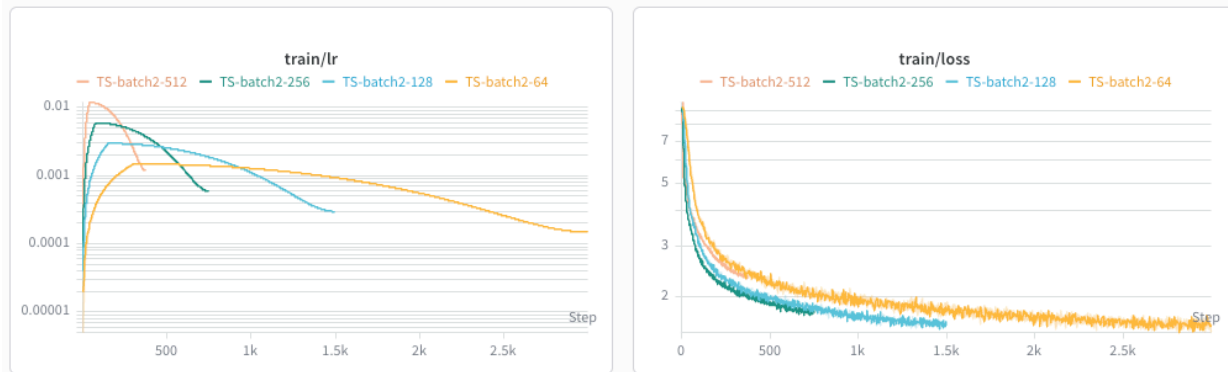
7.2 Batch Size Variations

Recall the relationship: $\text{total_tokens} = \text{batch_size} * \text{context_size} * \text{iterations}$. We ran experiments for a fixed set of tokens: 15% of the training set. This means that we had to adjust the steps, as well as T_w/T_c . We also adjusted $a_{\min}$ and $a_{\max}$ (discussed below).

We note some interesting behaviors:

- batch_size does NOT affect wall-clock time (64 - 11m13s, 128 - 10m14s, 256 - 10m38s, 512 - 10m48s).
- batch sizes of 64 and 128 seem to do equally well, and they do noticeably better than 256. 512 does the worst by far.
- As we increase batch_size, more and more of the VRAM is filled.





At first, these things seem contradictory: as we fill the VRAM, and we are batching (=parallelizing in my mind) operations, why does the wall-clock time not change? This comes from a misunderstanding on my part of what's really happening with batching.

First of all, when running in a batch of 64, the matmuls necessary in the training steps are already parallelized across the GPU so that they are using all the GPU's compute units. In other words, FLOPs are the same, no matter how we batch. This is not the same as VRAM, which tells us how much of the memory is being used to store activations. (This obviously gets higher as we increase the batch size, as we need to track more activations at the same time, but is not the bottleneck here.) Instead what's happening when we increase batch size, is that we are taking a larger sample of data to update our weights, meaning they will generally be less noisy weight updates.

Why do batch sizes of 64 and 128 do better than the larger batch sizes? It would seem to suggest that smaller, rougher updates do a better job than bigger, less noisy updates. The intuition behind this might be that more frequent rougher updates are more nimble and better able to navigate the complex loss space.

We scaled learning rate ($a_{\max}$) as we adjusted iteration size ("as we take fewer steps, we need to take bigger steps to go the same distance"). However, this relationship may work for smaller learning rates but at larger learning rates this relationship breaks down simply because large learning rates are more likely to cause instability.

LLM suggestion: The more precise point is that total FLOPs are constant (same tokens, same model), so the GPU just takes the same total time to chew through them regardless of batching. You could simplify to: "Total FLOPs depend on total

tokens processed and model size, both of which are constant across runs, so wall-clock time is constant."

7.2 Generating Text

temp: 0.0, p: 0.3

First Try:

Hank was so excited that he wanted to show everyone his new trick. He swam around the pond, showing off his new trick. Everyone was amazed by how clever he was.

Hank was so proud of himself. He had made a new friend and he was so proud of himself. He was so happy that he had been able to show everyone how clever he was.

The end.

<endoftext>

Second Try:

Hank was so excited that he wanted to show everyone his new trick. He swam around the pond, showing off his new trick. Everyone was amazed by how clever he was.

Hank was so proud of himself. He had made a new friend and he was so proud of himself. He was so happy that he had been able to show everyone how clever he was.

The end.

<endoftext>

temp: 0.8, p: 0.9

First Try:

And they all lived happily ever after.

<endoftext>

Second Try:

Hank wanted to have some fun with the other ducks. He said, "Hey, don't worry, I can sell you to you. Let's get some food and a nice house."

Hank smiled and said, "Yes! Let's do that. I like to have new friends and make some money."

So they went to the store and bought some food. They also bought some bread and butter. The fat duck was so happy. He thanked Hank for being so nice and the other ducks were happy too.

The end.

<endoftext>

Third try:

One day, Hank decided to take a nap in the pond. He snored and snooze until he heard a loud splash.

Hank opened his eyes and saw a baby duck swimming towards him. The baby duck said, "Hank, can you help me? I need to find my mom."

Hank smiled and said, "Sure! I'll help you find your mom."

So, Hank and the baby duck went on a hunt together. Hank watched over the pond and found the baby duck's mom.

He told her, "I'm sorry, Hank. I was just so tired. I can't help now."

Hank smiled and said, "That's okay. I'll be there to help."

And that's what Hank did. He swam around the pond and finally, he was so proud of himself.

<endoftext>

temp: 0.0, p: 0.9

First Try:

Hank was so excited that he wanted to show everyone his new trick. He swam around the pond, showing off his new trick. Everyone was amazed by how clever he was.

Hank was so proud of himself. He had made a new friend and he was so proud of himself. He was so happy that he had been able to show everyone how clever he was.

The end.

<endoftext>

Second Try:

Hank was so excited that he wanted to show everyone his new trick. He swam around the pond, showing off his new trick. Everyone was amazed by how clever he was.

Hank was so proud of himself. He had made a new friend and he was so proud of himself. He was so happy that he had been able to show everyone how clever he was.

The end.

<endoftext>

temp: 0.8, p: 0.3

First Try:

One day, Hank and Hank went to the pond to swim. Hank said to Hank, "Let's go swimming!" Hank said, "No, let's stay here and play!" Hank was so sad that he wanted to go home.

But Hank said, "Don't be sad Hank. We can still have fun!" So Hank and Hank had lots of fun swimming together.

The end.

<endoftext>

Second Try:

Hank was so excited that he wanted to take a closer look at the duck. He swam closer and closer until the duck was right next to him!

The duck was very friendly and asked Hank if he wanted to play with him. Hank said yes and they started to play together.

Hank and the duck had lots of fun together, and Hank was very happy.

When it was time to go, the fat duck said goodbye and swam away.

Hank was sad to see the duck go, but he knew he would come back soon.

<endoftext>

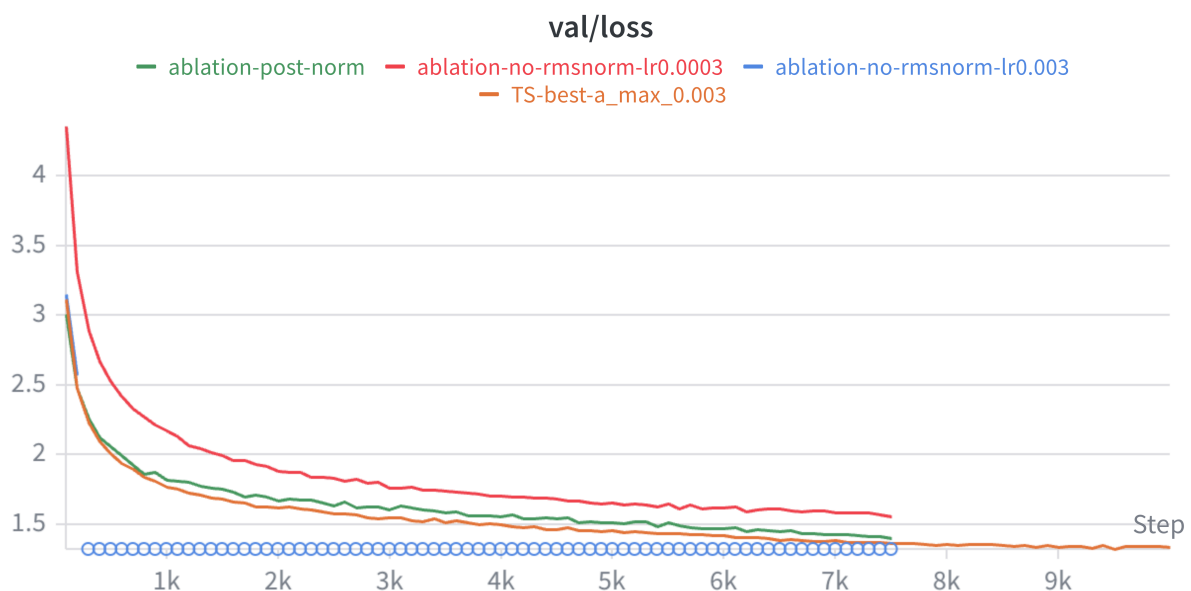
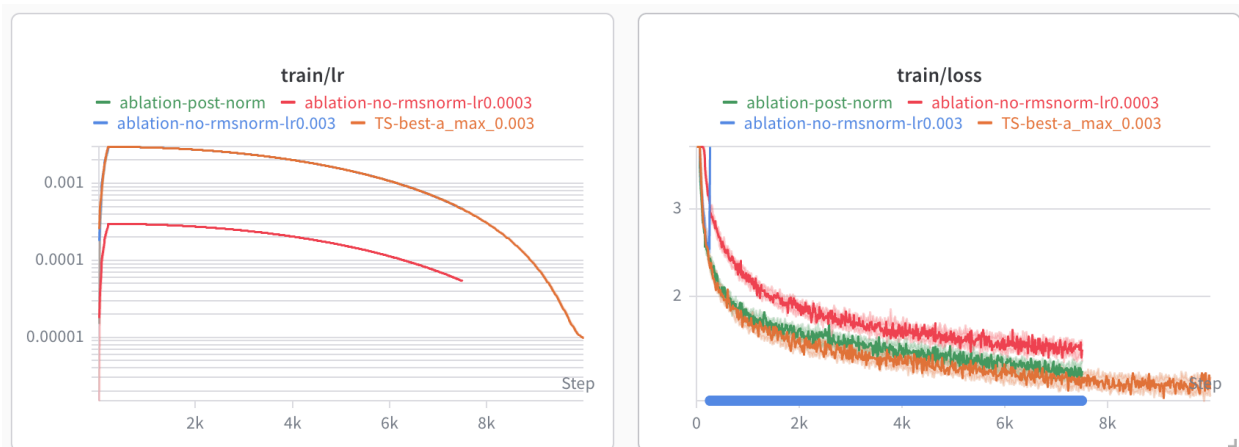
Prompt: There once was a fat duck named Hank.

We note that in the examples above, a temperature of zero produces exactly the same output, independent of the p. This is exactly what we would expect.

It's much harder to tell the difference between a top-p value of 0.3 and 0.9, for the

same temperature. In theory, we should expect that a p-value of 0.3 means that the model will “play it safe”; it’s not necessarily picking the top word every time, but it makes this pool of words its choosing from very small, so the word choices are usually safer. (In the case where the top word has a pred probability greater than 0.3, then it will take the top word.) With this framing, it seems like you can start to see some patterns here. The words in the p-val of 0.3 case are much simpler and more directly relevant to the initial prompt.

7.3 Ablations: Drop RMSNorm and Implement Post Norm



When we remove RMS norm (blue), our initial computations work but we quickly get Nan's from our model. Why? No idea. I think it could have something to do with vanishing gradients? Basically, we don't have anything "standardizing" or keeping our encodings in check, so they get way out of proportion.

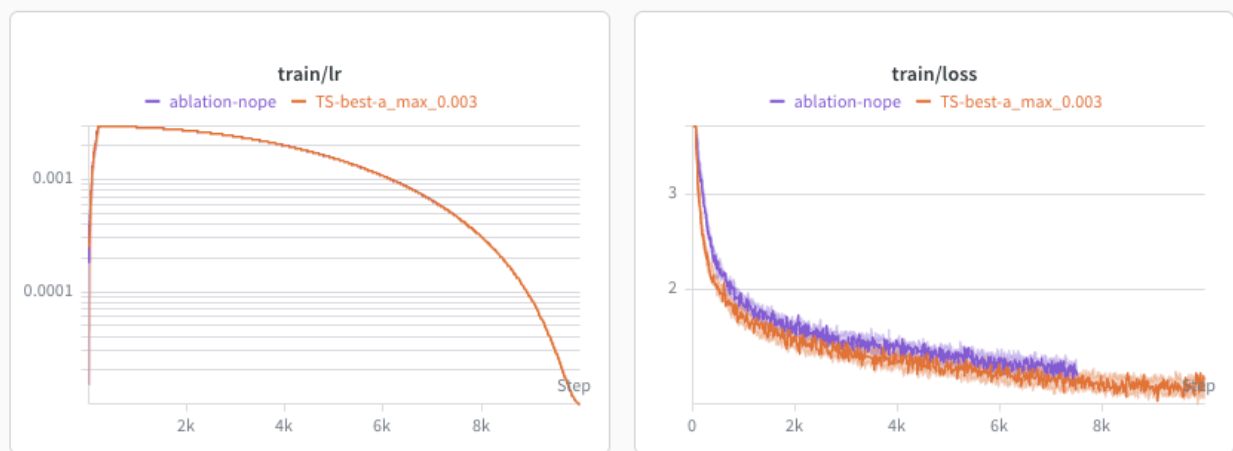
- **LLM commentary:** Actually, exploding gradients. Float32 has max value of $1e38$ or so, which means that if activations get above this, they become `inf`, which can lead to `Nan`'s downstream if we get any `inf*inf` or similar calculations. At the start of training, the weight values are still relatively low, but after roughly 100 steps they become too large. If we have a much lower learning rate, step sizes are much smaller, and we avoid any huge corrections which cause the gradients, and therefore activations huge. It avoids triggering instability that result in exploding gradients.

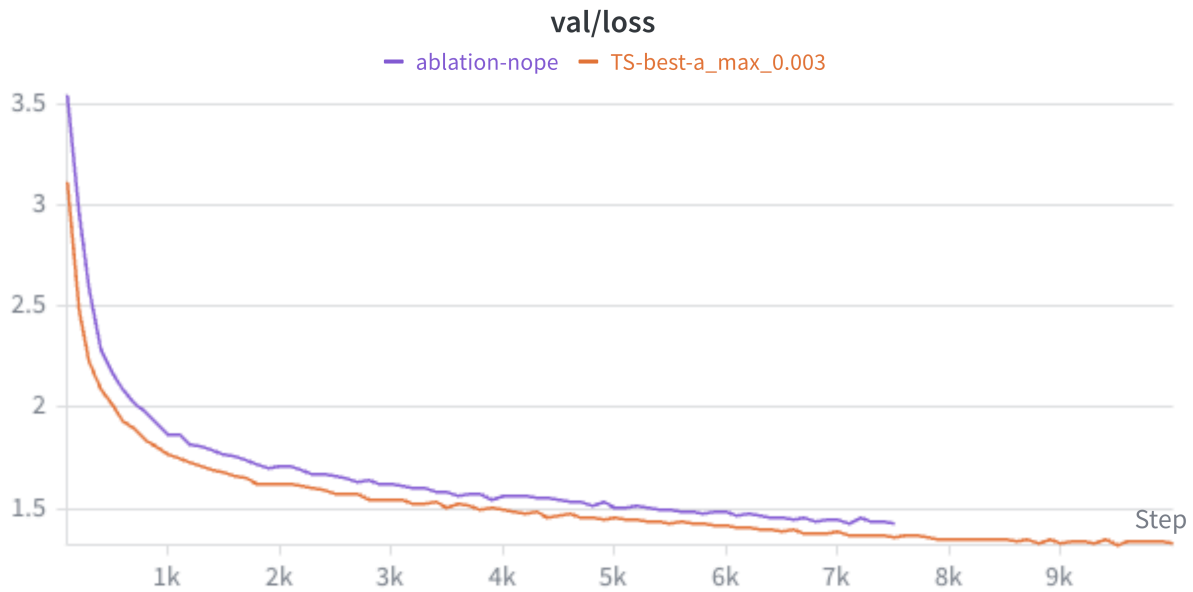
We CAN get stability from a significantly lower learning rate (0.0003), but we note that the results aren't as good (though to be fair we didn't experiment to find the optimal value here).

Finally, we note that by comparing the orange (pre-norm) and green lines (post-norm), our pre-norm architecture performs slightly better, as expected.

- **LLM commentary:** Pre-norm is better because gradients flow through the residual unimpeded.

7.3 Ablations: NoPE

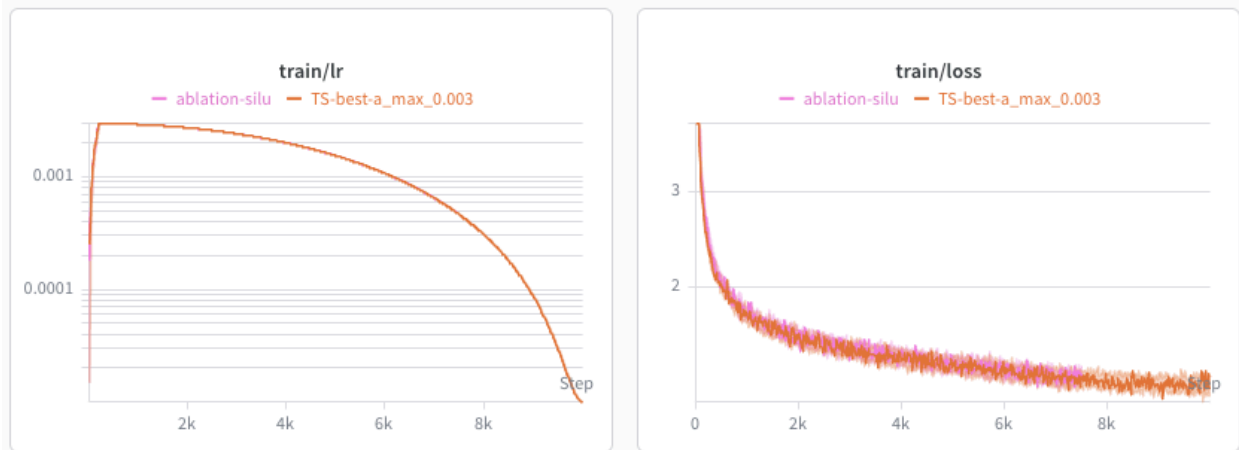




Here we see RoPE (orange) performs better than NoPE (purple). Surprisingly, NoPE does remarkably well! As the assignment sheet mentions, this must mean that they can still infer relative or absolute position information by some other means.

7.3 Ablations: SwiGLU vs. SuiLU





These were basically identical (looking at the perplexity, you can tell that the pink line is a hair higher). Not sure if I did something wrong, or if our dataset or hyperparameter combinations are such that there is not real difference in our case.

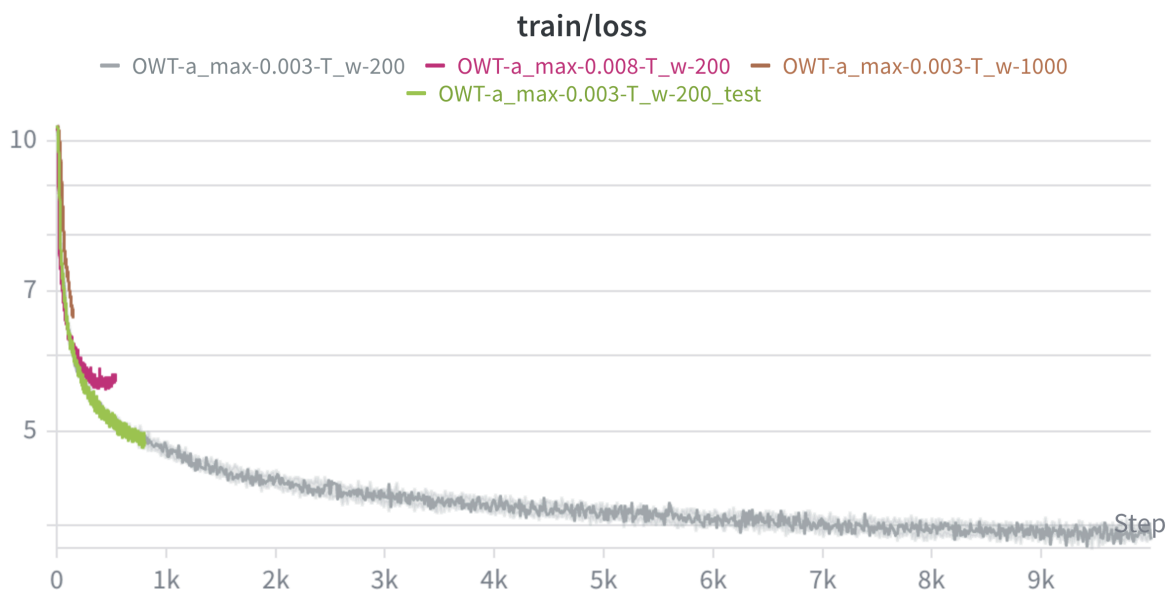
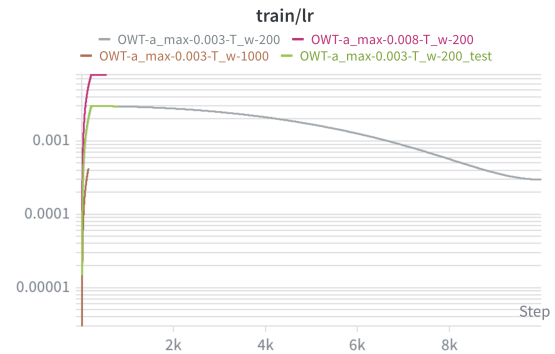
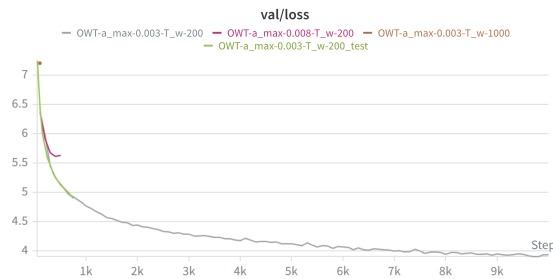
7.4 Open Web Text

A few notes:

- This took about 1h40m to train the 10k iterations; this is like ~40% longer than doing the same 10k iterations on the Tiny Stories dataset which took 1h7m. I'm assuming most of this comes from the fact that we have a 3x in the vocab size.
- Our initial $a_{\min}$ of 0.003 seemed to work best, though I didn't tinker too much with the params here.
- The losses are significantly higher for the same iterations, plateauing just below 4, whereas our Tiny Stories model got below 1.4. This large difference is also do to the vocabulary size but also the complexity of the dataset—the topics in the training set are just so much broader, therefore harder to learn, and therefore score worse. The reason a larger vocabulary might make it harder is that many of the words may come up infrequently, and therefore have few examples to learn from.

LLM comment: Yes, the 3x vocab size means the LM head (the final linear layer) is 3x larger — $(d_{\text{model}}=512) \times (\text{vocab_size}=32000)$ vs $\times 10000$. That matmul

happens every forward AND backward pass, so it's a big chunk of compute. The softmax over 32k classes is also more expensive.



7.5

Our OWT model has a <5 so we'll just leave it at that and go to the next assignment!